

Assignment

Objective

Here I am going to build the class conditioning transformer. So if I give the text cat on car it generate the shape stroke by stroke and with the Mixture Density Network (MDN)

Introduction

The Objective of this model is to generate hand-drawn sketches belonging to a specific class such as Airplane, Apple Car fish etc.

Unlike image generation models that generate pixels directly, the model generates sketches as a sequence of pen movements

Every sketch is represented as sequence of:

(dx, dy, pen_state)

where

- dx = movement in horizontal direction
- dy = movement in vertical direction
- pen_state = state of the pen.

Pen states are:

- 0 $\rightarrow$ pen touching the paper
- 1 $\rightarrow$ pen lifting the paper
- 2 $\rightarrow$ end of drawing.

The Architecture consists of

- Class Conditioning module
- Decode only transformer
- Mixture Density Network (MDN)
- pen state prediction Head

- The transformer Learns the Drawing Sequence while the MDN Learns the probability distribution of the next drawing movement.

Why Normal Regression Cannot be Used

- Suppose a person starts Drawing an Airplane Wing. At a particular point there may be multiple valid next directions
- Option 1 (move slightly left) Option 2 (move slightly right)
Option 3 (move upward) All of these movements to draw an image is correct
- If MSE is used, the model predicts the Average of all possibilities
- For example possible next points: (10,5) (20,10) (30,15)
mse prediction (20,10) this Average point may not correspond to any realistic drawing stroke
- As a result generated sketches become blurry, Strokes becomes unrealistic, drawing lose diversity.
- Therefore Instead of predicting a single coordinate the model predicts a probability distribution of possible coordinates

Understanding probability distributions

- The model predicts : what are the ~~possible~~^{all} possible next movements and how likely is each movement?
- Instead of saying next point (10,5) the model says there is 40% chance of (10,5) 35% chance of (15,8) 25% chance of (20,10)

Multi Variate Gaussian Distribution

- Gaussian distribution is also called as Normal distribution. It has μ, σ (center, spread) the values that are far from the center (μ) is less likely.
- The Stroke Generation model predicts (dx, dy) which contains 2 variables. The normal Gaussian handles only one variable. therefore we need a distribution that handles two variables simultaneously. This is called Multi variate Gaussian distribution.
- So therefore model predicts $P(dx, dy)$
- Mean vector: the center of the distribution is $\mu = [\mu_x, \mu_y]$
example: $\mu = [5, 3]$ this means most likely movement is $dx=5, dy=3$
- Covariance matrix: the model also learns the relationship between dx and dy . Suppose dx increases then dy whether it increases or decreases.
- Covariance matrix $\Sigma = \begin{bmatrix} \sigma_x^2 & \text{cov}(x, y) \\ \text{cov}(y, x) & \sigma_y^2 \end{bmatrix}$

where σ_x^2, σ_y^2 represent the spread of the distribution

How Multivariate Gaussian is implemented in the code

- There is a class Normal Network create the Gaussian parameters
- the code `mean = self.mean_net(x)` predicts μ the code `tril = self.tril_net(x)` predicts Σ
- the model therefore learns mean, covariance for every possible next stroke

- The output becomes

Multivariate Normal (mean, scale_tril = tril)

Why one Gaussian is Not Enough

- Suppose the model is drawing a tree At one position the next branch can go left, right, up A single Gaussian only represent one peak However there are multiple valid directions Therefore one Gaussian is insufficient

Mixture Density Network MDN

- To solve this problem the model uses multiple Gaussians. Here we used $n_components = 20$. therefore 20 Gaussian distributions are created.

- Each Gaussian represents one possible future drawing direction

the final probability becomes

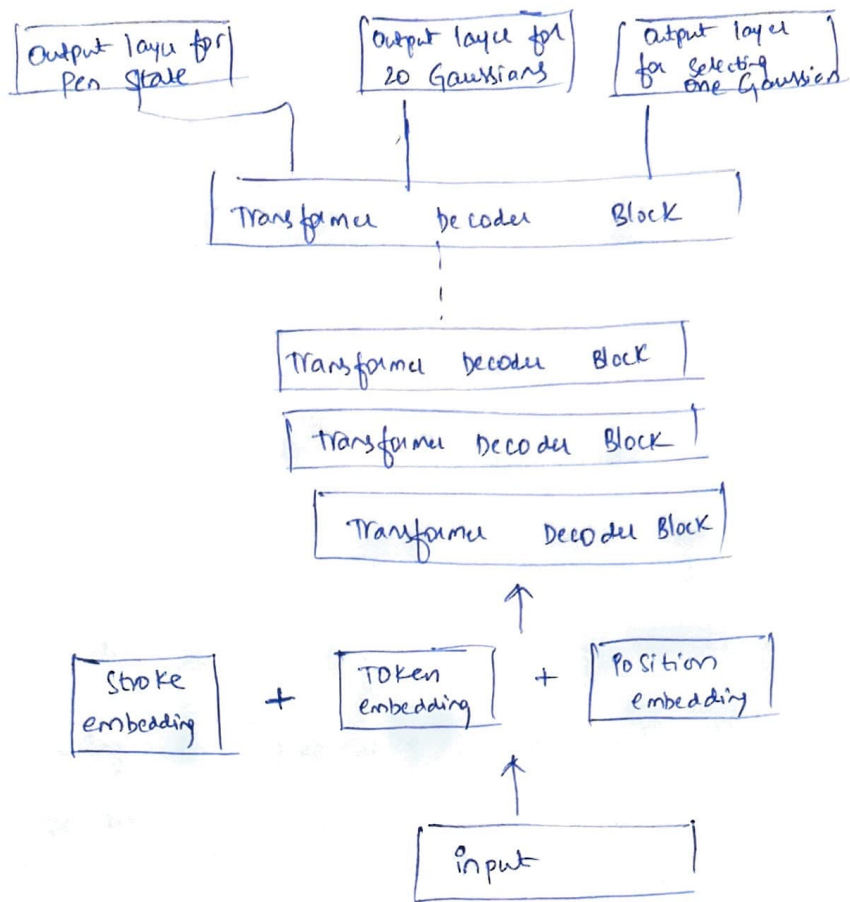
$$\text{probability} = \text{weight}_1 \times \text{Gaussian}_1 + \text{weight}_2 \times \text{Gaussian}_2 \\ \dots + \text{weight}_{20} \times \text{Gaussian}_{20}$$

- This combination is called a Mixture Density Network

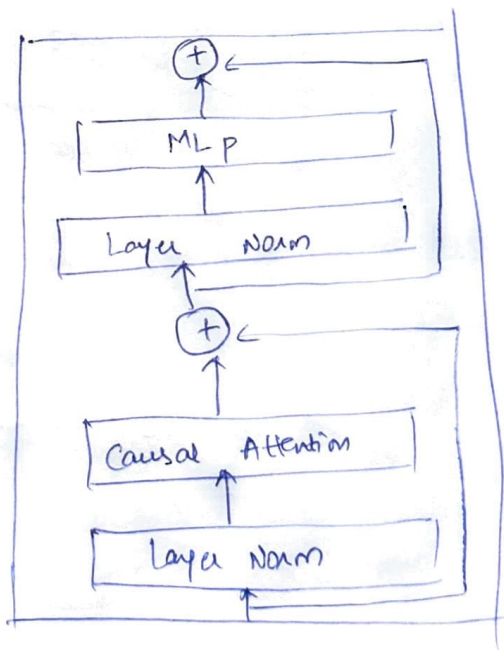
What MDN Predicts

- For every time step the MDN predicts
 1. mean of each Gaussian
 2. Covariance of each Gaussian
 3. Weight of each Gaussian

Architecture



Transformer Block



Code Explanation

- I have downloaded 10 classes like Airplane, apple, car, etc. npz files

- each npz files contain train, test, valid

- Example of npz

[

[5, 2, 0],

[3, 1, 0],

[-2, 4, 1],

]

Shape (L, 3)

L = number of strokes

- Columns dx, dy, penstate

penstate 3 values 0, 1, 2

0 $\Rightarrow$ pen on paper

1 $\Rightarrow$ Lift the pen from paper

2 $\Rightarrow$ end of sketch

- Let's take [5, 2, 0] dx=5 dy=2 penstate=0, this is move 5 right and 2 up

- The Output from the Dataset class I get Xs, Ys, mask, class_ids

- Xs is the Input Sequence shape [B, T, 3]

- Ys is the target Sequence shape [B, T, 3] shifted by one

- class_ids that stroke belong to which class its shape is (B,) (i.e. Aeroplane, car, ...)

- We have the Stroke embedding, pen embedding, position embedding, class embedding

- Now X_s input sequence is taken and I apply $X[:, :, 2]$ means I take 1st 2 columns that dx, dy so the shape become $[B, T, 2]$

I have the stroke embedding layer whose weights are $(2, 384)$ dx, dy each of 384 dimension

- So X_s become $[B, T, 384]$

- Coming to pen embedding we have '3' penstates Pen on paper, pen lift, end sketch so the embedding layer $(3, 384)$

- position embedding so T positions so embedding layer $[T, 384]$

- class embedding 10 classes because I downloaded 10 classes only so class embedding layer $[10, 384]$

- So $X = \text{stroke_emb} + \text{pen_emb} + \text{pos_emb} + \text{cls_emb}$

- the final $X = [B, T, 384]$

- So the input to transformer decoder block is $[B, T, 384]$

• Layer Norm

→ 384 Columns so $384 \times 2 = 768$ γ, β we take each Row of token

$$\rightarrow \mu = \frac{1}{384} \sum_{i=1}^{384} x_i$$

$$\sigma^2 = \frac{1}{384} \sum_{i=1}^{384} (x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

ϵ is very small value to avoid Division by zero

$$\rightarrow y = \gamma_i \hat{x}_i + \beta_i$$

→ if the model want to move to some other distribution then also it can move to some other distribution that's why γ, β introduced the output of Layer Norm $[B, T, 384]$

• Transformer

→ num_heads = 6 so 384 dim so each head dim = 64

~~$$\rightarrow W_q = [64, 64] \quad W_k = [64, 64] \quad W_v = [64, 64]$$~~

~~$$\Rightarrow Q = X \cdot W_q \Rightarrow [B, T, 64]$$~~

$$\rightarrow W_q = [384, 384] \quad W_k = [384, 384] \quad W_v = [384, 384]$$

$$\rightarrow Q = X \cdot W_q = [B, T, 384] \cdot [384, 384] = [B, T, 384]$$

$$K = X \cdot W_k = [B, T, 384] \cdot [384, 384] = [B, T, 384]$$

$$V = X \cdot W_v = [B, T, 384] \cdot [384, 384] = [B, T, 384]$$

→ Q is reshaped as $[B, 6, T, 64]$

K is reshaped as $[B, 6, T, 64]$

V is Reshaped as $[B, 6, T, 64]$

→ The formula

$$\text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d}} \right) \cdot V$$

$$\text{So } Q \cdot K^T \Rightarrow [B, 6, T, 64] \cdot [B, 6, 64, T] \\ = [B, 6, T, T]$$

$$\sqrt{d} = \sqrt{64} = 8$$

→ Before Applying Softmax All the future tokens are masked to $-\infty$ so $\text{softmax}(-\infty) = 0$

The shape $[B, 6, T, T]$

→ multiplied with V $[B, 6, T, 64]$

$$[B, 6, T, T] \cdot [B, 6, T, 64] = [B, 6, T, 64]$$

→ Again reshape to $[B, T, 384]$

→ the final projection layer in the Attention whose weights $[384, 384]$

$$[B, T, 384] \cdot [384, 384] = [B, T, 384]$$

• Residual Addition Applied the tensor before the Layer Norm $[B, T, 384]$ and the tensor After the Causal Attention

• Again the Layer Norm $[B, T, 384] \rightarrow [B, T, 384]$

768 trainable parameters

- MLP

$$[B, T, 384] \cdot [384, 1536] = [B, T, 1536] \quad \text{and Activation}$$

Relu Applied

$$[B, T, 1536] \cdot [1536, 384] = [B, T, 384]$$

- Again the Residual Connection

$$[B, T, 384] \rightarrow \text{before 2nd layer Norm}$$

$$[B, T, 384] \rightarrow \text{after MLP}$$

$$[B, T, 384] + [B, T, 384] = [B, T, 384]$$

- Final Layer Norm Applied only once when After all the Transformer blocks are done

768 trainable parameters

- Output Layer

→ 1st Neural Network is pi-net whose weights are

$$[384, 20]$$

$$[B, T, 384] \cdot [384, 20] = [B, T, 20] \quad \text{this gives the}$$

probability of each Gaussian Softmax Applied

→ 2nd Neural Network is for the multivariate Gaussian to predict mean & Covariance this 2nd NN has 2 Networks

1st Network $[384, 40]$

$$[B, T, 384] \cdot [384, 40] = [B, T, 40]$$

$[B, T, 40]$ reshape to $[B, T, 20, 2]$ to get 20 Gaussians (dx, dy)

2nd Network $[384, 60]$

$$[B, T, 384] \cdot [384, 60] = [B, T, 60]$$

$[B, T, 60]$ reshape to $[B, T, 20, 3]$ to get $\sigma_x^2, \sigma_y^2, \text{cov}(x, y)$

- 3rd Neural Network for the pen state

$$[B, T, 384] \cdot [384, 3] = [B, T, 3]$$

- there 3' outputs of 3' NN is given by the output layer $[B, T, 20]$, $[B, T, 20, 2]$ & $[B, T, 20, 3]$, $[B, T, 3]$

- the target that is described $[B, T, 3]$ Split the target in $[\cdot, \cdot, :2]$, $[\cdot, \cdot, :2]$
 $\Downarrow$
 $[B, T, 2]$, $[B, T]$

- Now there are 20 Gaussian and assume target (dx, dy)
 How likely the target (dx, dy) under Gaussian 1 Similarly
 How likely the target (dx, dy) under Gaussian 2 like that
 for 20 Gaussian 20 values

- So Now do element wise multiplication between Probability p-net and these 20 values add them to get result

- then $-\log(\text{result})$

$$L_{\text{Coordinate}} = -\log(P_{\text{mixture}})$$

- for the pen loss 3' categories so

Categorical

$$L_{\text{pen}} = -\sum_i y_i \log \hat{y}_i$$

- total loss = Coordinate-loss + pen loss
 $= L_{\text{Coordinate}} + L_{\text{pen}}$

- How the Given target belong to probability Gaussian

$$P(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)\right)$$

if the target is close to mean more probable
if the target far from mean less probable

Generation

- once the model is trained we start from $dx, dy = (0,0)$ pass through the model
get the next (dx, dy) and pen state
- Like this model draws stroke by stroke.

Parameters used to train model

- 1) batch_size = 64
- 2) training_iters = 50000
- 3) eval_interval = training_iters // 20 # for every 1200 steps
- 4) lr = $1e-4$
- 5) Cuda
- 6) embed = 384
- 7) embed_ffn = $384 \times 4 = 1536$
- 8) num_heads = 6
- 9) n_layers = 6
- 10) dropout = 0.2
- 11) n_components = 20 # using 20 gaussian